

Partición de Modelos de Gran Escala para Simulación en Paralelo por Eventos Discretos

Franco Sansone

Joaquín Fernández, Ernesto Kofman

Facultad de Ciencias Exactas, Ingeniería y Agrimensura - Universidad Nacional de Rosario.

Marzo 2025

Organización de la Presentación

- 1 Marco Teórico
- 2 Balance de Carga Usando Grafos Basados en Conjuntos
- 3 Resultados y Trabajo Futuro

Modelos Matemáticos

Definición de Modelo Matemático

Conjunto de expresiones matemáticas (ecuaciones, inecuaciones, ecuaciones diferenciales) que representan un sistema.

Simulación

La simulación de un modelo consiste en la ejecución del mismo a través del tiempo.

Modelos Matemáticos

Definición de Modelo Matemático

Conjunto de expresiones matemáticas (ecuaciones, inecuaciones, ecuaciones diferenciales) que representan un sistema.

Simulación

La simulación de un modelo consiste en la ejecución del mismo a través del tiempo.

Problema: Modelos complejos acarrear alto costo computacional

Modelos Matemáticos

Definición de Modelo Matemático

Conjunto de expresiones matemáticas (ecuaciones, inecuaciones, ecuaciones diferenciales) que representan un sistema.

Simulación

La simulación de un modelo consiste en la ejecución del mismo a través del tiempo.

Problema: Modelos complejos acarrear alto costo computacional

Solución: Simulación en Paralelo

Simulación en Paralelo

Los algoritmos de simulación en paralelo requieren que:

- El modelo original sea dividido en P submodelos.
- Cada submodelo sea simulado por un hilo diferente.

Simulación en Paralelo

Los algoritmos de simulación en paralelo requieren que:

- El modelo original sea dividido en P submodelos.
- Cada submodelo sea simulado por un hilo diferente.

Para obtener una paralelización efectiva:

- La **partición** del modelo debe ser **balanceada**.
- La **comunicación** entre los submodelos debe ser **minimizada**.

Simulación en Paralelo

Los algoritmos de simulación en paralelo requieren que:

- El modelo original sea dividido en P submodelos.
- Cada submodelo sea simulado por un hilo diferente.

Para obtener una paralelización efectiva:

- La **partición** del modelo debe ser **balanceada**.
- La **comunicación** entre los submodelos debe ser **minimizada**.

Requieren que **particionemos el modelo**.

Simulación en Paralelo

Los algoritmos de simulación en paralelo requieren que:

- El modelo original sea dividido en P submodelos.
- Cada submodelo sea simulado por un hilo diferente.

Para obtener una paralelización efectiva:

- La **partición** del modelo debe ser **balanceada**.
- La **comunicación** entre los submodelos debe ser **minimizada**.

Requieren que **particionemos el modelo**. Llamamos a este proceso *Balance de carga*.

Balance de Carga para Simulación en Paralelo

Podemos distinguir tres tipos de problemas de balance de carga:

- **Balance de carga estático:** la asignación a los procesadores se realiza al inicio de la simulación.
- **Balance de carga semi-estático:** la asignación a los procesadores ocurre al comienzo de la simulación, pero una conocimiento de dominio extra.
- **Balance de carga dinámico:** la asignación a los procesadores se hace dinámicamente en el transcurso de la simulación.

Balance de Carga para Simulación en Paralelo

Podemos distinguir tres tipos de problemas de balance de carga:

- **Balance de carga estático:** la asignación a los procesadores se realiza al inicio de la simulación.
- Balance de carga semi-estático: la asignación a los procesadores ocurre al comienzo de la simulación, pero una conocimiento de dominio extra.
- Balance de carga dinámico: la asignación a los procesadores se hace dinámicamente en el transcurso de la simulación.

Balance de Carga para Simulación en Paralelo

Podemos distinguir tres tipos de problemas de balance de carga:

- **Balance de carga estático:** la asignación a los procesadores se realiza al inicio de la simulación.
- Balance de carga semi-estático: la asignación a los procesadores ocurre al comienzo de la simulación, pero una conocimiento de dominio extra.
- Balance de carga dinámico: la asignación a los procesadores se hace dinámicamente en el transcurso de la simulación.

El problema de balance de carga se puede tratarse como un problema de **particionamiento de grafos**.

Métodos de Particionamiento de Grafos

● Métodos geométricos.

- Agrupan por coordenadas geométricas.
- Son rápidos y fáciles de implementar.
- Inducen costos altos de comunicación.

● Métodos combinatorios.

- Agrupan según la adyacencia del grafo.
- Particiones de mejor calidad.
- Más demandantes computacionalmente.

● Métodos espectrales.

- Calculan una distancia basada en conectividad y dividen balanceadamente.
- Produce particionamientos de alta calidad.
- Escalan ineficientemente cuando el tamaño del grafo crece.

● Métodos multinivel.

- Arma el grafo, compacta, refina y expande el grafo.
- Son lo más utilizados en esta área. Producen particiones de alta calidad.
- La compactación y expansión del grafo es más costosa cuando el grafo crece.

Métodos de Particionamiento de Grafos

● Métodos geométricos.

- Agrupan por coordenadas geométricas.
- Son rápidos y fáciles de implementar.
- Inducen costos altos de comunicación.

● Métodos combinatorios.

- Agrupan según la adyacencia del grafo.
- Particiones de mejor calidad.
- Más demandantes computacionalmente.

● Métodos espectrales.

- Calculan una distancia basada en conectividad y dividen balanceadamente.
- Produce particionamientos de alta calidad.
- Escalan ineficientemente cuando el tamaño del grafo crece.

● Métodos multinivel.

- Arma el grafo, compacta, refina y expande el grafo.
- Son lo más utilizados en esta área. Producen particiones de alta calidad.
- La compactación y expansión del grafo es más costosa cuando el grafo crece.

¿Cómo construimos dicho grafo?

Grafo Computacional

Las unidades de cálculo y sus dependencias se pueden modelar de forma natural mediante un **grafo no dirigido**.

Más específicamente, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, l, w)$, donde:

$\mathcal{V}$ el conjunto de nodos/las unidades de computación más pequeñas

$\mathcal{E} : \mathcal{E} \subset \mathcal{V} \times \mathcal{V}$ el conjunto de aristas/dependencias entre unidades

$l : \mathcal{V} \rightarrow \mathcal{R}^+$ asigna un peso computacional a cada nodo

$w : \mathcal{E} \rightarrow \mathcal{R}^+$ asigna un peso comunicacional a cada arista

Este grafo se conoce como **Grafo Computacional**.

Grafo Computacional

Las unidades de cálculo y sus dependencias se pueden modelar de forma natural mediante un **grafo no dirigido**.

Más específicamente, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, l, w)$, donde:

$\mathcal{V}$ el conjunto de nodos/las unidades de computación más pequeñas

$\mathcal{E} : \mathcal{E} \subset \mathcal{V} \times \mathcal{V}$ el conjunto de aristas/dependencias entre unidades

$l : \mathcal{V} \rightarrow \mathcal{R}^+$ asigna un peso computacional a cada nodo

$w : \mathcal{E} \rightarrow \mathcal{R}^+$ asigna un peso comunicacional a cada arista

Este grafo se conoce como **Grafo Computacional**.

Necesitamos una forma de obtener dicho grafo computacional $\mathcal{G}$ de un modelo implementado en **Modelica**.

Modelo Advección-Difusión-Reacción

Lo siguiente es un fragmento de una implementación en Modelica de un modelo Advección-Difusión-Reacción (ADR):

```

1 Real u[N];
2 ...
3 der(u[1])=-a*(u[1]-1)/dx+d*(-2*u[1]+1)/(dx^2)+r*(u[1]^2)*(1-u[1]);
4 for i in 2:N loop
5     der(u[i])=-a*(u[i]-u[i-1])/dx+ d*(-2*u[i]+u[i-1])/(dx^2)+
6         r*(u[i]^2)*(1-u[i]);
7 end for;

```

Queremos poder construir su grafo computacional a partir de su implementación.

Modelo Advección-Difusión-Reacción

Lo siguiente es un fragmento de una implementación en Modelica de un modelo Advección-Difusión-Reacción (ADR):

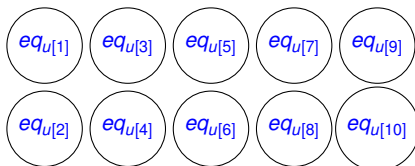
```

1 Real u[N];
2 ...
3 der(u[1])=-a*(u[1]-1)/dx+d*(-2*u[1]+1)/(dx^2)+r*(u[1]^2)*(1-u[1]);
4 for i in 2:N loop
5     der(u[i])=-a*(u[i]-u[i-1])/dx+ d*(-2*u[i]+u[i-1])/(dx^2)+
6     r*(u[i]^2)*(1-u[i]);
7 end for;

```

Queremos poder construir su grafo computacional a partir de su implementación.

Ejemplo para $N = 10$.



Modelo Advección-Difusión-Reacción

Lo siguiente es un fragmento de una implementación en Modelica de un modelo Advección-Difusión-Reacción (ADR):

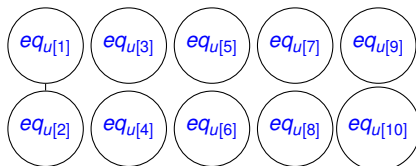
```

1 Real u[N];
2 ...
3 der(u[1])=-a*(u[1]-1)/dx+d*(-2*u[1]+1)/(dx^2)+r*(u[1]^2)*(1-u[1]);
4 for i in 2:N loop
5     der(u[i])=-a*(u[i]-u[i-1])/dx+ d*(-2*u[i]+u[i-1])/(dx^2)+
6     r*(u[i]^2)*(1-u[i]);
7 end for;

```

Queremos poder construir su grafo computacional a partir de su implementación.

Ejemplo para $N = 10$.



Modelo Advección-Difusión-Reacción

Lo siguiente es un fragmento de una implementación en Modelica de un modelo Advección-Difusión-Reacción (ADR):

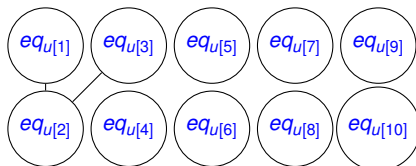
```

1 Real u[N];
2 ...
3 der(u[1])=-a*(u[1]-1)/dx+d*(-2*u[1]+1)/(dx^2)+r*(u[1]^2)*(1-u[1]);
4 for i in 2:N loop
5     der(u[i])=-a*(u[i]-u[i-1])/dx+ d*(-2*u[i]+u[i-1])/(dx^2)+
6     r*(u[i]^2)*(1-u[i]);
7 end for;

```

Queremos poder construir su grafo computacional a partir de su implementación.

Ejemplo para $N = 10$.



Modelo Advección-Difusión-Reacción

Lo siguiente es un fragmento de una implementación en Modelica de un modelo Advección-Difusión-Reacción (ADR):

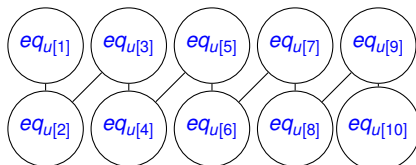
```

1 Real u[N];
2 ...
3 der(u[1])=-a*(u[1]-1)/dx+d*(-2*u[1]+1)/(dx^2)+r*(u[1]^2)*(1-u[1]);
4 for i in 2:N loop
5     der(u[i])=-a*(u[i]-u[i-1])/dx+ d*(-2*u[i]+u[i-1])/(dx^2)+
6     r*(u[i]^2)*(1-u[i]);
7 end for;

```

Queremos poder construir su grafo computacional a partir de su implementación.

Ejemplo para $N = 10$.



Mejoras en el Balance de Carga

Los algoritmos **multinivel** crean el grafo, compactan, mejoran las particiones iniciales y expanden el grafo nuevamente.

Queremos evitar dicha compactación y posterior expansión, buscamos aprovechar la naturaleza cíclica de **Modelica**.

Veamos una estructura de datos que nos permite representar grafos en forma compacta.

Grafos Basados en Conjuntos

Un Grafo Basado en Conjuntos (SBG) no dirigido es una tupla $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \text{map}_1, \text{map}_2)$ donde:

- $\mathcal{V} = \{V_1, \dots, V_n\}$ es un conjunto de vértices-conjunto disjuntos.
- $\mathcal{E} = \{E_1, \dots, E_m\}$ es un conjunto de aristas-conjunto que une vértices-conjunto de $\mathcal{V}$.
- $\text{map}_1 : \mathcal{E} \rightarrow \mathcal{V}$ tal que dado $e \in \mathcal{E}$, $\text{map}_1(e)$ representa uno de los extremos de la arista e .
- $\text{map}_2 : \mathcal{E} \rightarrow \mathcal{V}$ tal que dado $e \in \mathcal{E}$, $\text{map}_2(e)$ representa el otro de los extremos de la arista e .

Grafos Basados en Conjuntos

Sea $G = (V, E)$ un grafo tradicional no dirigido donde:

- $V = \{1, \dots, 10\}$ son sus vértices-conjunto.
- $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 5\}, \{5, 6\}, \{6, 7\}, \{7, 8\}, \{8, 9\}, \{9, 10\}\}$ son sus aristas-conjunto.

Una definición equivalente del mismo usando un SBG

$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \text{map}_1, \text{map}_2)$ donde:

- $\mathcal{V} = \{[1 : 1], [2 : 10]\}$ son sus nodos.
- $\mathcal{E} = \{[1 : 1], [2 : 9]\}$ son sus aristas.
- Y sus mapas pueden definirse por medio de funciones por partes

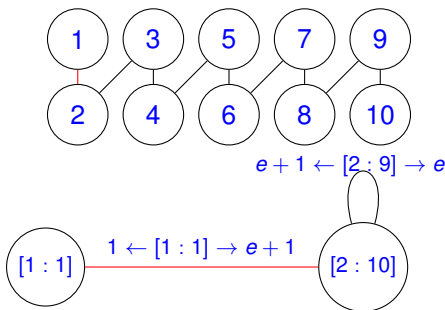
$$\text{map}_1(e) = \begin{cases} 1 & \text{si } e \in [1 : 1] \\ e + 1 & \text{si } e \in [2 : 9] \end{cases}$$

$$\text{map}_2(e) = \begin{cases} e + 1 & \text{si } e \in [1 : 1] \\ e & \text{si } e \in [2 : 9] \end{cases}$$

Grafos Basados en Conjuntos

En la descripción gráfico del grafo $\mathcal{G}$, o su equivalente $\mathcal{G}$, definido previamente:

- las aristas correspondientes al intervalo $[1 : 1]$ están en color rojo.
- las correspondientes al intervalo $[2 : 9]$ en negro.



Grafos Basados en Conjuntos y Modelica



¿Cómo podríamos construir un grafo computacional basado en conjuntos a partir de un modelo implementado en Modelica?

Organización de la Presentación

- 1 Marco Teórico
- 2 Balance de Carga Usando Grafos Basados en Conjuntos
- 3 Resultados y Trabajo Futuro

Motivación

Los métodos de particionamiento actuales son sensibles al tamaño del grafo.

Modelos de gran escala → grafos computacionales de gran tamaño.

Queremos representar el grafo computacional en forma compacta y particionarlo sin necesidad de expandir.

La implementación del modelo tiene una estructura compacta, ¿podemos aprovecharla?

Motivación

Los métodos de particionamiento actuales son sensibles al tamaño del grafo.

Modelos de gran escala → grafos computacionales de gran tamaño. Queremos representar el grafo computacional en forma compacta y particionarlo sin necesidad de expandir.

La implementación del modelo tiene una estructura compacta, ¿podemos aprovecharla?

Presentamos un algoritmo de Balance de Carga usando Grafos Basados en Conjuntos.

Formato de Entrada

Si definimos la ocurrencia de una variable en una expresión dada como:

Algorithm 1 Tipo Var

Var: {

id : string

$cost$: int

exp : (a, b) donde $a, b \in N^0$ y estos representan constante y pendiente en la expresión $a \times i + b$

def : $\{n_1, \dots, n_k\}$ donde n_i es el id de un conjunto de nodos donde se calcula la variable. Es decir, aparece en el lado izquierdo de la ecuación.

}

Formato de Entrada

Un nodo individual que representa una unidad de cálculo en el modelo puede ser definido así:

Algorithm 2 Tipo Node

```
Node: {  
  id: string  
  weight: int  
  interval:  $[1, \dots, M]$   
  lhs:  $\{L_1, \dots, L_k\}$  donde  $L_i$  tiene tipo Var  
  rhs :  $\{R_1, \dots, R_g\}$  donde  $R_i$  tiene tipo Var  
}
```

Y finalmente podemos definir: **Nodes** : $\{I_1, \dots, I_m\}$ con I_j del tipo **Node** como entrada del algoritmo de particionado.

Modelo Advección-Difusión-Reacción

Recordemos la implementación del Modelo ADR:

```

1 Real u[N];
2 ...
3 der(u[1])=-a*(u[1]-1)/dx+d*(-2*u[1]+1)/(dx^2)+r*(u[1]^2)*(1-u[1]);
4 for i in 2:N loop
5     der(u[i])=-a*(u[i]-u[i-1])/dx+ d*(-2*u[i]+u[i-1])/(dx^2)+
6     r*(u[i]^2)*(1-u[i]);
7 end for;

```

¿Cómo podríamos representar este modelo usando el tipo `Nodes`?

Representación del Modelo ADR

```

1 Real u[N];
2 ...
3 der(u[1])=-a*(u[1]-1)/dx+d*(-2*u[1]+1)/(dx^2)+r*(u[1]^2)*(1-u[1]);
4 for i in 2:N loop
5     der(u[i])=-a*(u[i]-u[i-1])/dx+ d*(-2*u[i]+u[i-1])/(dx^2)+
6     r*(u[i]^2)*(1-u[i]);
7 end for;

```

Nos centraremos en la línea 5.

$$V_3 = \{$$

id : *u*

cost = 1

exp = (1, 0) → para formar la expresión

$1 \times i + 0$ que corresponde a *u*[*i*]

defs = {}

$$\}$$

Figura 1: Lado izquierdo de la expresión en línea 5.

Representación del Modelo ADR

```

1 Real u[N];
2 ...
3 der(u[1])=-a*(u[1]-1)/dx+d*(-2*u[1]+1)/(dx^2)+r*(u[1]^2)*(1-u[1]);
4 for i in 2:N loop
5     der(u[i])=-a*(u[i]-u[i-1])/dx+ d*(-2*u[i]+u[i-1])/(dx^2)+
6     r*(u[i]^2)*(1-u[i]);
7 end for;

```

Nos centraremos en la línea 5.

$$V_4 = \{$$

id : u

cost = 1

exp = $(1, 0) \rightarrow$ para formar la expresión

$1 \times i + 0$ que corresponde a $u[i]$

defs = $\{1, 2\}$

$\}$

Figura 1: Lado derecho de la expresión en línea 5.

$$V_5 = \{$$

id : u

cost = 1

exp = $(1, -1) \rightarrow$ para formar la expresión

$1 \times i - 1$ que corresponde a $u[i - 1]$

defs = $\{1, 2\}$

$\}$

Figura 2: Lado derecho de la expresión en línea 5.

Representación del Modelo ADR

```

1 Real u[N];
2 ...
3 der(u[1])=-a*(u[1]-1)/dx+d*(-2*u[1]+1)/(dx^2)+r*(u[1]^2)*(1-u[1]);
4 for i in 2:N loop
5     der(u[i])=-a*(u[i]-u[i-1])/dx+ d*(-2*u[i]+u[i-1])/(dx^2)+
6     r*(u[i]^2)*(1-u[i]);
7 end for;

```

Nos centraremos en la línea 5.

$$l_2 = \{$$

$$id : 2$$

$$weight = 1$$

$$interval = [2, N]$$

$$lhs = \{V_3\}$$

$$rhs = \{V_4, V_5\}$$

$$\}$$

Figura 1: Expresión en línea 5.

Representación del Modelo ADR

La línea 3 puede representarse de la siguiente manera:

$$V_1 = \{$$

id = *u*

cost = 1

exp = (0, 1) → para formar la expresión

$0 \times i + 1$ que corresponde a *u*[1]

defs = {}

$$\}$$

Figura 2: Lado izquierdo de la expresión de la línea 3

$$V_2 = \{$$

id : *u*

cost = 1

exp = (0, 1) → para formar la expresión

$0 \times i + 1$ que corresponde a *u*[1]

defs = {1, 2}

$$\}$$

Figura 3: Lado derecho de la expresión en línea 3

Representación del Modelo ADR

La línea 3 puede representarse de la siguiente manera:

$$l_1 = \{$$

$$id : 1$$

$$weight = 1$$

$$interval = [1, 1]$$

$$lhs = \{V_1\}$$

$$rhs = \{V_2\}$$

$$\}$$

Figura 4: Expresión de tipo Node correspondiente a la línea 3 del modelo

Finalmente, el Modelo ADR quedaría representado por la expresión $l :$
 $Nodes = [l_1, l_2]$.

Creación del grafo computacional

Los nodos del grafo se obtienen de la siguiente manera, sea $I \in$

Nodes:

$$\mathcal{V} = \{\forall n \in I : n.\text{interval}\}$$

Nuestro grafo computacional $\mathcal{G}$ para el modelo ADR hasta el momento se ve así:



Creación del grafo computacional

Los nodos del grafo se obtienen de la siguiente manera, sea $I \in$

Nodes:

$$\mathcal{V} = \{\forall n \in I : n.\text{interval}\}$$

Nuestro grafo computacional $\mathcal{G}$ para el modelo ADR hasta el momento se ve así:



Necesitamos construir las aristas de $\mathcal{G}$.

Creación del grafo computacional



Sea $R_2 = l_2.rhs = \{V_4, V_5\}$.

V_5 está definida en l_1 y l_2 .

Veamos para el caso l_1 :

Sea $D = lm(\{[2 : N]\}, i - 1) \cap lm(\{[1 : 1]\}, 1)$. Si $D \neq \emptyset$, debemos construir una arista entre l_2 y l_1 .

$D = \{[1 : N - 1]\} \cap \{[1 : 1]\}$ (asumimos $N > 2$)

$D = \{[1 : 1]\}$

$\therefore$ hay conexión, debemos construir una arista-conjunto de cardinalidad $\#D$ entre l_2 y l_1 .

Creación del grafo computacional

Debemos definir una parte de las funciones map_1 y map_2 , las que conecten los nodos de l_2 y l_1 :

La expresión de $V_1.lhs$ es constante, por lo tanto la función es:

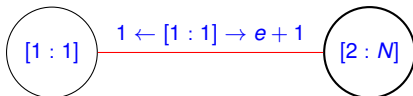
$map_1(e) = 1$ para $e \in \{[1 : 1]\}$.

La expresión de V_5 es $(1, -1)$. Para estos casos el mapa lineal lo obtenemos a partir de $var.exp - D_{offset} + V_{offset}$:

$map_2(e) = e - 1 + 2 = e + 1$ para $e \in \{[1 : 1]\}$

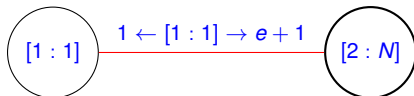
Creación del grafo computacional

Hasta el momento el Grafo Computacional Basado en Conjuntos se ve así:

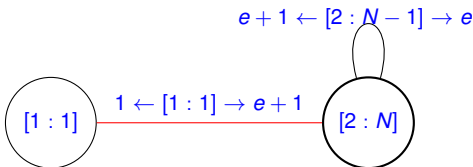


Creación del grafo computacional

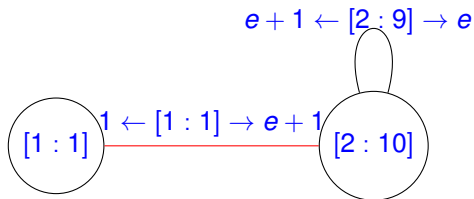
Hasta el momento el Grafo Computacional Basado en Conjuntos se ve así:



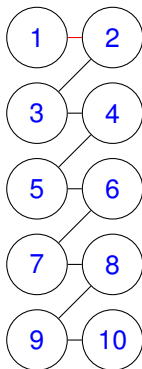
Al completar la búsqueda de conexiones para $R_2 = \{V_4, V_5\}$ el grafo se ve así:



Grafo Computacional del Modelo ADR



(a) Grafo computacional SBG para $N = 10$



(b) Grafo computacional expandido correspondiente

Particionamiento Inicial

Debemos hacer un **particionamiento inicial**

Se elige un **nodo raíz**.

Se recorre el grafo por anchura.

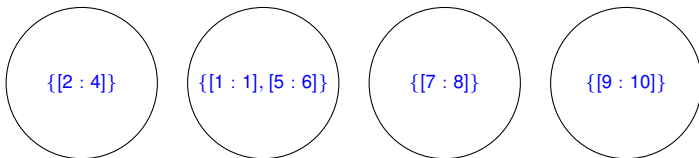
Usamos dos estrategias para el armado de las particiones iniciales.

- **Estrategia Greedy**, a medida que se recorre el grafo, se agregarán todos los elementos a una partición hasta que esté completa.
- **Estrategia Distributiva**, cada intervalo se parte en P partes iguales y se reparten una parte para cada partición.

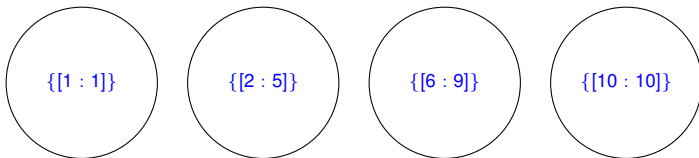
Particionamiento Inicial del Modelo ADR

Ejemplos de particionado inicial para el grafo computacional basado en conjuntos del Modelo ADR.

Usando la estrategia distributiva con recorrido en pre orden:



Usando la estrategia greedy con recorrido en post orden:



Refinamiento

Una vez obtenidas las particiones iniciales, entramos en la etapa de refinamiento, etapa en la cual buscamos disminuir la comunicación entre las mismas, sin desbalancear.

Refinamiento

Una vez obtenidas las particiones iniciales, entramos en la etapa de refinamiento, etapa en la cual buscamos disminuir la comunicación entre las mismas, sin desbalancear.

Buscamos disminuir el *edgeCut*.

Refinamiento

Una vez obtenidas las particiones iniciales, entramos en la etapa de refinamiento, etapa en la cual buscamos disminuir la comunicación entre las mismas, sin desbalancear.

Buscamos disminuir el *edgeCut*.

Dada una partición $\mathcal{C}$ en P particiones disjuntas $\mathcal{V} = (\mathcal{C}_1 \cup \dots \cup \mathcal{C}_p)$,

$$cost(\mathcal{C}) = edgeCut(\mathcal{C}) = \sum_{\mathcal{C}_i, \mathcal{C}_j} \{w(u, v) | u \in \mathcal{C}_i, v \in \mathcal{C}_j, i \neq j\},$$

Kernighan-Lin

Sea S un conjunto de $2n$ puntos, con una matriz de costo asociada $C = (c_{ij})$, $i, j = 1, \dots, 2n$.

Queremos partir S en dos conjuntos A y B , cada una con n puntos, tales que el “costo externo” $T = \sum_{A \times B} c_{ab}$ se minimice.

Definamos para cada $a \in A$, un *costo externo* (EC) E_a

$$E_a = \sum_{y \in B} c_{ay}$$

y un *costo interno* (IC) I_a

$$I_a = \sum_{x \in A} c_{ax}$$

Análogamente, definimos E_b , I_b para cada $b \in B$.

Kernighan-Lin

Sea

$$D_x = E_x - I_x \text{ para todo } x \in S;$$

D_x es la diferencia entre costo externo e interno.

Considerando cualquier $a \in A$, $b \in B$. Si a y b son intercambiados, la **ganancia** (esto es, la reducción del costo) es precisamente.

$$D_a + D_b - 2c_{ab}$$

Este proceso se repite y se acumulan las ganancias parciales. Una vez culminado, se elige el intercambio con ganancia máxima.

Kernighan-Lin

Fase de optimización para dos particiones:

- Computar D para todos los elementos de S .
- Elegir $a_i \in A$, $b_j \in B$ tal que: $g_1 = D_{a_i} + D_{b_j} - 2c_{a_i b_j}$ es máxima.
- Recalcular D para los elementos de $A - \{a_i\}$ y $B - \{b_j\}$
- Repetir el paso dos, eligiendo un par a'_2 , b'_2 tal que:
 $g_2 = D_{a'_2} + D_{b'_2} - 2c_{a'_2 b'_2}$ es máxima.
- Repetir el proceso hasta que se hayan agostado todos los nodos.

Si $X = a'_1, a'_2, \dots, a'_k$, $Y = b'_1, b'_2, \dots, b'_k$, entonces la disminución en costo al intercambiar X e Y es precisamente $g_1 + g_2 + \dots + g_k$.

Elegir k que maximice la suma parcial $\sum_{i=1}^k g_i = G$.

Si $G > 0$, habrá una reducción al intercambiar X e Y .

Kernighan-Lin

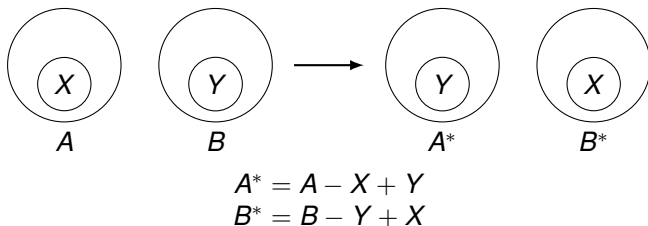


Figura 6: Descripción gráfica de la fase de optimización.

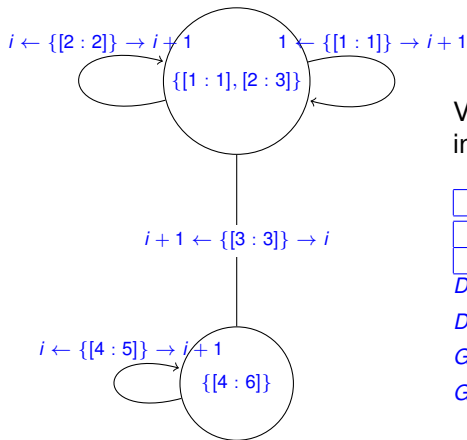
Cálculo de EC e IC en el Particionador SBG

- S , intervalo o conjunto sobre el cual calcular EC e IC.
- P que es la partición a la cual pertenece S .
- Q , que es la otra partición que estamos refinando.
- map_1 y map_2 son los mapas del grafo.

Algorithm 3 Compute EC & IC

```
1: function COMPUTE_EC_IC( $S, P, Q, map_1, map_2$ )
2:    $D = map_1.prelimage(S)$ 
3:    $I = map_2.image(D)$ 
4:    $IC' = (P \cap I) / S$ 
5:    $IC = map_2.prelimage(IC') \cap D$ 
6:    $EC' = (I / IC') \cap Q$ 
7:    $EC = map_2.prelimage(EC') \cap D$ 
8:   return ( $EC, IC$ )
9: end function
```

Ejemplo de Optimización de Particiones



Veamos la ganancia de intercambiar $[1 : 1]$ con $[4 : 6]$.

Node	EC	IC
$\{[1 : 1]\}$	$\emptyset$	$\{[1 : 1]\}$
$\{[4 : 6]\}$	$\{[3 : 3]\}$	$\emptyset$

$$D_{[1:1]} = \#EC_{[1:1]} - \#IC_{[1:1]} = 0 - 1 = -1$$

$$D_{[4:6]} = \#EC_{[4:6]} - \#IC_{[4:6]} = 1 - 0 = 1$$

$$Gain = D_{[1:1]} + D_{[4:6]} - 2 * C_{[1:1],[4:6]}$$

$$Gain = -1 + 1 - 2 * 0 = 0$$

Particionador SBG

Presentamos nuestro algoritmo de optimización de particiones:

Algorithm 4 Algoritmo de optimización Kernighan-Lin para SBG

```

1: function KL-SBG( $G, P = (A, B), C$ )
2:    $A_c = A$  ▷ Hacer una copia de las particiones originales
3:    $B_c = B$ 
4:   ... ▷ Inicializar variables
5:    $GM = \text{compute\_gain\_matrix}(A_c, B_c, G, C)$  ▷ Computar matriz de ganancia
6:   while  $A_c \neq \emptyset \wedge B_c \neq \emptyset$  do ▷ Mientras queden elementos en ambas particiones, seguir iterando
7:      $\langle i, j, g, s \rangle = \text{max\_diff}(GM)$  ▷ Obtener intercambio con mayor ganancia
8:      $(A', B') = \text{update\_sets}(A_c, B_c, A_v, B_v, i, j, s)$  ▷ Actualizar conjuntos
9:      $\text{update\_sum}(par\_sum, g, max\_par\_sum, max\_par\_sum\_set, A', B')$  ▷ Actualizar suma parcial
10:     $\text{update\_gain\_matrix}(A_c, B_c, i, j, s, GM, G)$  ▷ Actualizar las copias  $A_c$  y  $B_c$ 
11:  end while
12:  ...
13: end function

```

KL-SBG para múltiples particiones

Sea $P = \{P_1, \dots, P_n\}$ la partición inicial de nuestro grafo, queremos comprobar si podemos mejorarla.

- Calcular las posibles ganancias:

$$\forall i \in \{1, \dots, n\} \forall j \in \{i + 1, \dots, n\} G_{ij} = KL - SBG(P_i, P_j)$$

- Tomamos la ganancia máxima G_{ij} . Si $G_{ij} > 0$, efectuamos el intercambio.

- Sean $X \subset P_i$, $Y \subset P_j$ los conjuntos cuyo intercambio genera ganancia G_{ij} , entonces:

$$P'_i = P_i \setminus X \cup Y$$

$$P'_j = P_j \setminus Y \cup X.$$

- Repetir desde el primer paso con las particiones i y j actualizadas hasta que no haya ganancias positivas.

KL-SBG para múltiples particiones

Algunas mejoras:

- Aplicar más de un intercambio cuando sea posible siguiendo la siguiente regla:
 - Sea $G = \{G_{xy}, \dots\}$ el conjunto de ganancias positivas luego de aplicar $KL - SBG$ a las particiones.
 - Sea G_{ij} la ganancia máxima, realizar los intercambios para P_i y P_j .
 - Descartar todas las ganancias que involucren a P_i y P_j ,
 $G' = G \setminus G_{xy}$ tal que $x \in \{i, j\} \vee y \in \{i, j\}$.
 - Repetir hasta que $G' = \emptyset$.
- Aplicar $KL - SBG$ entre dos particiones P_i, P_j si existe comunicación entre las mismas, es decir, si $edgeCut(P_i, P_j) > 0$.
- Las ejecuciones de $KL - SBG$ se ejecutan en paralelo.

Organización de la Presentación

- 1 Marco Teórico
- 2 Balance de Carga Usando Grafos Basados en Conjuntos
- 3 Resultados y Trabajo Futuro

Métricas de particionado

Métricas de calidad de particionado:

- **Edge-Cut:** la suma del peso de las aristas que conectan nodos entre particiones.
- **Volumen de comunicación total:** la cantidad de comunicación entre particiones.
- **Volumen de comunicación máximo:** calcula el volumen de comunicación máximo entre particiones.
- **Desbalance máximo:** calcula el desbalance máximo entre particiones.

Métricas de particionado

Métricas de rendimiento:

- **Tiempo de Inicialización:** es el tiempo que le toma al particionador en leer la entrada y construir el grafo de conexiones.
- **Tiempo de Particionado:** Es el tiempo que le toma al algoritmo generar las particiones iniciales y optimizarlas.

METIS implementa de métodos multinivel y será utilizado para comparar los resultados obtenidos.

Modelo ADR

Dado que las variables que comunican datos entre las distintas particiones son $u[i]$ con $u[i-1]$ para un determinado rango. El *edgeCut* es igual a 3, pues es la arista que comunica los dos nodos adyacentes entre particiones independientemente del tamaño.

Particionador	Inicialización (ms)	Particionado (ms)	Edge cut	Communication volume	Maximum volume	Maximum imbalance
SBG-Partitioner	0,4	5,31	3	6	2	0
METIS	2,9	1,2	3	6	2	1,28e2

Cuadro 1: Modelo con tamaño 10000

Particionador	Inicialización (ms)	Particionado (ms)	Edge cut	Communication volume	Maximum volume	Maximum imbalance
SBG-Partitioner	0,5	5,51	3	6	2	0
METIS	26	12	3	6	2	1,76e-2

Cuadro 2: Modelo con tamaño 100000

Modelo Aires Acondicionados con Controlador

Este modelo representa una población de aires acondicionados, dividida por secciones, cada sección conectada a un controlador parcial, y estos conectados a un controlador centralizado.

Particionador	Inicialización (ms)	Particionado (ms)	Edge cut	Communication volume	Maximum volume	Maximum imbalance
SBG-Partitioner	2	7275	75004	150005	74999	1,3e-5
METIS	182	105	149925	149927	75004	1,3e-5

Cuadro 3: Modelo con tamaño 100000

Particionador	Inicialización (ms)	Particionado (ms)	Edge cut	Communication volume	Maximum volume	Maximum imbalance
SBG-Partitioner	3	7360	750004	1500005	749999	1,3e-6
METIS	14741	1501	1500003	1500004	749973	6,5e-5

Cuadro 4: Modelo con tamaño 1000000

Trabajo Futuro

- Implementación:
 - Actualizar librería SBG a su última versión.
 - Evitar recomputar conexiones innecesariamente (estado global).
 - Posibilitar el desbalance de particiones.
- Mejorar $KL - SBG$ para N particiones usando las componentes conexas del grafo, no sólo aristas individuales.
- Implementar algoritmos de balance de carga semi-estático y dinámico usando SBG.

